```
import re
import io
import os
import base64
import streamlit as st
import plotly.graph_objects as go

st.set_page_config(page_title="CVSync AI | INTELLIQ SOLVEX", page_icon="🎯",
layout="wide",
                   initial_sidebar_state="collapsed")


def dark(fig):
    fig.update_layout(template="plotly_dark", paper_bgcolor="rgba(0,0,0,0)",
plot_bgcolor="rgba(0,0,0,0)")
    return dark(fig)

# ---------------------------------------------------------------
# SKILL DATABASE: canonical name -> (category, [synonyms])
# ---------------------------------------------------------------
SKILLS = {
    # Languages
    "Python": ("Programming Languages", ["python", "python3"]),
    "Java": ("Programming Languages", ["java", "core java"]),
    "C++": ("Programming Languages", ["c++", "cpp"]),
    "C": ("Programming Languages", ["c programming", "c language"]),
    "JavaScript": ("Programming Languages", ["javascript", "js", "ecmascript"]),
    "TypeScript": ("Programming Languages", ["typescript"]),
    "R": ("Programming Languages", ["r programming", "r language"]),
    "Go": ("Programming Languages", ["golang"]),
    "Scala": ("Programming Languages", ["scala"]),
    "Bash/Shell": ("Programming Languages", ["bash", "shell scripting", "shell"]),
    # Databases
    "SQL": ("Databases", ["sql", "t-sql", "pl/sql"]),
    "MySQL": ("Databases", ["mysql"]),
    "PostgreSQL": ("Databases", ["postgresql", "postgres"]),
    "MongoDB": ("Databases", ["mongodb", "mongo"]),
    "Redis": ("Databases", ["redis"]),
    "SQLite": ("Databases", ["sqlite"]),
    "Firebase": ("Databases", ["firebase", "firestore"]),
    # Data / ML
    "Machine Learning": ("Data Science & ML", ["machine learning", "ml"]),
    "Deep Learning": ("Data Science & ML", ["deep learning", "dl", "neural network",
"neural networks"]),
    "NLP": ("Data Science & ML", ["nlp", "natural language processing"]),
    "Computer Vision": ("Data Science & ML", ["computer vision", "cv", "image
```

```
processing", "opencv"]),
    "Generative AI / LLMs": ("Data Science & ML", ["generative ai", "genai", "llm",
"llms", "large language model", "large language models", "langchain", "rag"]),
    "Statistics": ("Data Science & ML", ["statistics", "statistical analysis",
"probability"]),
    "Data Analysis": ("Data Science & ML", ["data analysis", "data analytics", "eda",
"exploratory data analysis"]),
    "Data Visualization": ("Data Science & ML", ["data visualization", "data
visualisation", "matplotlib", "seaborn", "plotly"]),
    "Feature Engineering": ("Data Science & ML", ["feature engineering"]),
    # Libraries / Frameworks
    "Pandas": ("Libraries & Frameworks", ["pandas"]),
    "NumPy": ("Libraries & Frameworks", ["numpy"]),
    "Scikit-learn": ("Libraries & Frameworks", ["scikit-learn", "sklearn", "scikit
learn"]),
    "TensorFlow": ("Libraries & Frameworks", ["tensorflow", "tf", "keras"]),
    "PyTorch": ("Libraries & Frameworks", ["pytorch", "torch"]),
    "Hugging Face": ("Libraries & Frameworks", ["hugging face", "huggingface",
"transformers"]),
    "FastAPI": ("Libraries & Frameworks", ["fastapi", "fast api"]),
    "Flask": ("Libraries & Frameworks", ["flask"]),
    "Django": ("Libraries & Frameworks", ["django"]),
    "Streamlit": ("Libraries & Frameworks", ["streamlit"]),
    "React": ("Libraries & Frameworks", ["react", "reactjs", "react.js"]),
    "Node.js": ("Libraries & Frameworks", ["node.js", "nodejs", "node js",
"express.js", "expressjs"]),
    "Spring Boot": ("Libraries & Frameworks", ["spring boot", "springboot",
"spring"]),
    "HTML/CSS": ("Libraries & Frameworks", ["html", "css", "html5", "css3"]),
    "REST APIs": ("Libraries & Frameworks", ["rest api", "rest apis", "restful",
"restful api", "api development"]),
    # Tools / Cloud / DevOps
    "Docker": ("Tools, Cloud & DevOps", ["docker", "containerization",
"containers"]),
    "Kubernetes": ("Tools, Cloud & DevOps", ["kubernetes", "k8s"]),
    "AWS": ("Tools, Cloud & DevOps", ["aws", "amazon web services", "ec2", "s3
bucket", "sagemaker"]),
    "Azure": ("Tools, Cloud & DevOps", ["azure"]),
    "GCP": ("Tools, Cloud & DevOps", ["gcp", "google cloud"]),
    "Git/GitHub": ("Tools, Cloud & DevOps", ["git", "github", "gitlab", "version
control"]),
    "CI/CD": ("Tools, Cloud & DevOps", ["ci/cd", "cicd", "jenkins", "github
actions"]),
    "Linux": ("Tools, Cloud & DevOps", ["linux", "unix", "ubuntu"]),
    "Power BI": ("Tools, Cloud & DevOps", ["power bi", "powerbi"]),
    "Tableau": ("Tools, Cloud & DevOps", ["tableau"]),
    "Excel": ("Tools, Cloud & DevOps", ["excel", "advanced excel"]),
    "MLOps": ("Tools, Cloud & DevOps", ["mlops", "mlflow", "model deployment"]),
    "Spark/Hadoop": ("Tools, Cloud & DevOps", ["spark", "pyspark", "hadoop"]),
    # CS fundamentals / soft
```

```
    "Data Structures & Algorithms": ("CS Fundamentals", ["data structures",
"algorithms", "dsa"]),
    "Operating Systems": ("CS Fundamentals", ["operating systems", "operating
system"]),
    "OOP": ("CS Fundamentals", ["oop", "oops", "object oriented", "object-
oriented"]),
    "DBMS": ("CS Fundamentals", ["dbms", "database management"]),
    "Agile/Scrum": ("CS Fundamentals", ["agile", "scrum"]),
    "Communication": ("Soft Skills", ["communication", "communication skills"]),
    "Teamwork": ("Soft Skills", ["teamwork", "team player", "collaboration",
"collaborative"]),
    "Problem Solving": ("Soft Skills", ["problem solving", "problem-solving",
"analytical thinking"]),
}

# short, ambiguous tokens should only match in clean context
EDU_PATTERNS = {
    "B.E / B.Tech": r"\b(b\.?\s?e\.?|b\.?\s?tech|bachelor(?:'s)?(?: of
(?:engineering|technology))?)\b",
    "M.E / M.Tech": r"\b(m\.?\s?e\.?|m\.?\s?tech|master(?:'s)?(?: of
(?:engineering|technology|science))?)\b",
    "Computer/AI/DS Degree": r"(computer science|information technology|artificial
intelligence|data science|ai\s*&\s*ds|aids|cse)",
}

SAMPLE_JD = """Job Title: Machine Learning Engineer (Fresher / Entry Level)

We are looking for a Machine Learning Engineer with 0-2 years of experience to join
our AI team.

Requirements:
- Bachelor's degree in Computer Science, AI & Data Science or related field
- Strong programming skills in Python and SQL
- Hands-on experience with Machine Learning, Pandas, NumPy and Scikit-learn
- Experience building and deploying APIs using FastAPI or Flask
- Knowledge of Docker and cloud platforms such as AWS
- Familiarity with Git/GitHub and CI/CD
- Understanding of Deep Learning (TensorFlow or PyTorch) and NLP is a plus
- Good communication skills and problem solving ability
"""

SAMPLE_RESUME = """SRI KUMAR
Chennai, India | sri@example.com | github.com/sri

EDUCATION
B.Tech in Artificial Intelligence and Data Science, 2023 - 2027
CGPA: 8.4

SKILLS
Python, SQL, Machine Learning, Pandas, NumPy, Scikit-learn, Matplotlib, Flask, Git,
```

```
HTML, CSS

PROJECTS
Student Performance Predictor
- Built a Machine Learning model using Scikit-learn and Pandas to predict exam
results with 87% accuracy.
- Created a Flask web app to serve predictions and stored data in MySQL.

Sentiment Analysis on Tweets
- Applied NLP techniques with Python and TensorFlow to classify tweet sentiment.
- Visualised insights using Matplotlib and Seaborn.

EXPERIENCE
Data Analysis Intern, XYZ Technologies (2 months)
- Cleaned and analysed sales data using Python, Pandas and Excel.
- Collaborated with a team of 4 and presented findings to the manager.

ACHIEVEMENTS
- Participated in college hackathon, strong problem solving and communication skills.
"""


SAMPLE_BATCH = [
    "Python, SQL, Pandas, NumPy, Machine Learning, Scikit-learn. Project: built a
loan prediction model using Scikit-learn and Pandas. Built Flask API. Git, GitHub.
B.Tech Artificial Intelligence and Data Science. Good communication skills.",
    "Java, Spring Boot, MySQL, HTML, CSS, JavaScript. Project: Library management
system in Java with MySQL. Internship at ABC Soft. B.E Computer Science. Teamwork.",
    "Python, Machine Learning, Deep Learning, TensorFlow, NLP, Docker, AWS EC2,
FastAPI, Git, GitHub, CI/CD with GitHub Actions. Project: deployed a sentiment
analysis API with FastAPI and Docker on AWS. B.Tech Artificial Intelligence and Data
Science. Problem solving.",
    "C, C++, Data Structures and Algorithms, Operating Systems, Linux, DBMS, SQL.
Project: file system simulator in C++. B.E Computer Science and Engineering.",
    "Python, Pandas, NumPy, Matplotlib, Power BI, Excel, SQL, Statistics, Data
Analysis. Project: sales dashboard in Power BI. Intern at DataCorp (Data Analysis).
B.Tech Artificial Intelligence and Data Science. Communication skills.",
    "Python, PyTorch, Deep Learning, Computer Vision, OpenCV, Machine Learning,
Pandas, NumPy, Git. Project: face mask detection using PyTorch and OpenCV. B.Tech
Artificial Intelligence and Data Science. Problem solving, teamwork.",
]

# ---------------------------------------------------------------
# ROADMAP data
# ---------------------------------------------------------------
PROJECT_IDEAS = {
    "Docker": "Dockerize your existing Flask/FastAPI project. Write a Dockerfile,
build the image, run it and push to Docker Hub.",
    "AWS": "Deploy your ML model API on an AWS EC2 free-tier instance and store the
model file in an S3 bucket.",
```

```
    "FastAPI": "Convert your ML model into a FastAPI service with a /predict endpoint
and auto-generated Swagger docs.",
    "Flask": "Build a small Flask app with a form that takes input and returns an ML
prediction.",
    "SQL": "Design a 4-table database for a library system and write 15 queries
(joins, group by, subqueries).",
    "PostgreSQL": "Migrate a small project from SQLite to PostgreSQL and write 10
queries with joins and indexes.",
    "Machine Learning": "Predict house prices end to end: EDA, feature engineering, 3
models, cross-validation, report.",
    "Deep Learning": "Train a CNN on MNIST/CIFAR-10 and compare against a simple
baseline model.",
    "NLP": "Build a resume or review text classifier using TF-IDF and then a
transformer model.",
    "Generative AI / LLMs": "Build a PDF question-answering chatbot using RAG
(embeddings + vector store + LLM).",
    "CI/CD": "Add a GitHub Actions workflow that runs tests and lints your project on
every push.",
    "Git/GitHub": "Put 2 projects on GitHub with a proper README, branches and at
least 20 meaningful commits.",
    "Kubernetes": "Deploy a Dockerized app on Minikube with a Deployment and
Service.",
    "TensorFlow": "Train an image classifier with Keras and export it as a
SavedModel.",
    "PyTorch": "Implement a simple neural network training loop in PyTorch on a
tabular dataset.",
    "Pandas": "Take a messy Kaggle CSV and clean, merge and summarise it into a 1-
page insights report.",
    "NumPy": "Implement linear regression from scratch with NumPy only (gradient
descent).",
    "Scikit-learn": "Build a full sklearn Pipeline with preprocessing, GridSearchCV
and model comparison.",
    "Power BI": "Build a 3-page interactive dashboard on a public dataset (sales,
attendance, etc).",
    "Tableau": "Build a story dashboard on a public dataset and publish it to Tableau
Public.",
    "MLOps": "Track experiments with MLflow and package the best model with a simple
serving script.",
    "REST APIs": "Build a CRUD REST API for a to-do app with proper status codes and
validation.",
    "Linux": "Practice 30 core commands, write 3 shell scripts that automate file and
log tasks.",
    "Statistics": "Run hypothesis tests (t-test, chi-square) on a real dataset and
explain the findings.",
}

DAY_TEMPLATE = [
    "Understand what {s} is, why companies use it, and its core concepts. Watch one
crash course.",
    "Install / set up {s} and follow the official 'Getting Started' guide hands-on.",
```

```
    "Learn the 5-6 most used features of {s} and practise with small exercises.",
    "Solve 5-10 practice problems or tasks involving {s}.",
    "Start the mini-project (below). Build the basic working version.",
    "Finish and improve the mini-project. Add a README with screenshots.",
    "Push to GitHub, add it to your resume under Projects/Skills, and prepare 3
interview answers on {s}.",
]

LEVEL_WORDS = {
    "strong": (80, 101, "🟢 Strong Alignment"),
    "good": (60, 80, "🟡 Good Alignment"),
    "partial": (35, 60, "🟠 Partial Alignment"),
    "low": (0, 35, "🔴 Low Alignment"),
}

# ---------------------------------------------------------------
# Core helpers
# ---------------------------------------------------------------

def extract_pdf_text(file_bytes: bytes) -> str:
    text = ""
    try:
        import pdfplumber
        with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
            for page in pdf.pages:
                text += (page.extract_text() or "") + "\n"
    except Exception:
        text = ""
    if not text.strip():
        try:
            from pypdf import PdfReader
            reader = PdfReader(io.BytesIO(file_bytes))
            for page in reader.pages:
                text += (page.extract_text() or "") + "\n"
        except Exception:
            pass
    return text


def term_regex(term: str) -> re.Pattern:
    # custom boundaries so c++, node.js, ci/cd work properly
    return re.compile(r"(?<![A-Za-z0-9+#])" + re.escape(term) + r"(?![A-Za-z0-9+#])",
re.IGNORECASE)


def find_skills(text: str) -> set:
    found = set()
    for skill, (_, syns) in SKILLS.items():
        for term in set([skill.lower()] + syns):
            if skill in ("C", "R", "Go") and term == skill.lower():
```

```
                continue  # too ambiguous as a bare letter/word
            if term_regex(term).search(text):
                found.add(skill)
                break
    return found


def split_lines(text: str):
    lines = []
    for raw in re.split(r"[\n\r]+", text):
        raw = raw.strip(" -•*\t")
        if len(raw) > 3:
            lines.append(raw)
    return lines


def evidence_for(skill: str, resume_text: str, limit: int = 2):
    syns = set([skill.lower()] + SKILLS[skill][1])
    out = []
    for line in split_lines(resume_text):
        for term in syns:
            if skill in ("C", "R", "Go") and term == skill.lower():
                continue
            if term_regex(term).search(line):
                out.append(line)
                break
        if len(out) >= limit:
            break
    return out


def parse_years(text: str):
    """Returns (min_years, max_years) or None from JD text."""
    m = re.search(r"(\d+)\s*[-–to]+\s*(\d+)\s*\+?\s*years?", text, re.IGNORECASE)
    if m:
        return int(m.group(1)), int(m.group(2))
    m = re.search(r"(\d+)\s*\+?\s*years?", text, re.IGNORECASE)
    if m:
        return int(m.group(1)), int(m.group(1))
    if re.search(r"\bfresher|entry[- ]level|graduate\b", text, re.IGNORECASE):
        return 0, 1
    return None


def resume_experience_signals(text: str) -> dict:
    return {
        "internship": bool(re.search(r"\bintern(ship)?\b", text, re.IGNORECASE)),
        "projects": len(re.findall(r"\bproject", text, re.IGNORECASE)),
        "work": bool(re.search(r"\b(experience|worked|employed|engineer at|developer
at)\b", text, re.IGNORECASE)),
```

```
    }


def detect_role(jd: str) -> str:
    m = re.search(r"(?:job title|role|position)\s*[:\-]\s*(.+)", jd, re.IGNORECASE)
    if m:
        return m.group(1).strip()
    first = jd.strip().split("\n")[0].strip()
    return first[:80] if first else "Not specified"


def detect_edu(text: str) -> set:
    return {k for k, p in EDU_PATTERNS.items() if re.search(p, text, re.IGNORECASE)}


def classify_jd_skills(jd: str, jd_skills: set):
    """Split JD skills into 'required' and 'preferred (plus)' using the line they
appear in."""
    preferred, required = set(), set()
    for skill in jd_skills:
        lines = [l for l in split_lines(jd) if any(
            term_regex(t).search(l) for t in set([skill.lower()] + SKILLS[skill][1])
            if not (skill in ("C", "R", "Go") and t == skill.lower()))]
        is_pref = bool(lines) and all(
            re.search(r"\b(plus|preferred|nice to have|good to
have|bonus|desirable|optional|familiar)\b", l, re.IGNORECASE)
            for l in lines)
        (preferred if is_pref else required).add(skill)
    return required, preferred


def analyse(resume: str, jd: str) -> dict:
    jd_skills = find_skills(jd)
    res_skills = find_skills(resume)
    required, preferred = classify_jd_skills(jd, jd_skills)

    matched_req = required & res_skills
    matched_pref = preferred & res_skills
    missing_req = required - res_skills
    missing_pref = preferred - res_skills
    extra = res_skills - jd_skills

    # weighted skill score: required weight 1.0, preferred weight 0.5
    total_w = len(required) * 1.0 + len(preferred) * 0.5
    got_w = len(matched_req) * 1.0 + len(matched_pref) * 0.5
    skill_score = (got_w / total_w * 100) if total_w else 0

    # experience alignment
    yrs = parse_years(jd)
    sig = resume_experience_signals(resume)
```

```
    exp_score, exp_note = 70, "No clear experience requirement found in JD."
    if yrs is not None:
        if yrs[1] <= 1:
            exp_score = 100 if (sig["projects"] or sig["internship"]) else 60
            exp_note = f"JD expects fresher level ({yrs[0]}-{yrs[1]} yrs). Resume
shows " + (
                "projects/internship — good fit." if exp_score == 100 else "little
project/internship proof.")
        else:
            exp_score = 85 if sig["work"] and sig["internship"] else 55 if
(sig["internship"] or sig["work"]) else 35
            exp_note = f"JD asks ~{yrs[0]}-{yrs[1]} years. Resume shows " + (
                "some work/internship experience." if exp_score >= 55 else "mostly
academic projects.")

    # education alignment
    edu_resume = detect_edu(resume)
    edu_jd = detect_edu(jd)
    if edu_jd:
        edu_score = 100 if (edu_resume & edu_jd) else 50
        edu_note = "Education requirement matches." if edu_score == 100 else
"Education requirement not clearly visible in resume."
    else:
        edu_score, edu_note = 80, "No clear education requirement in JD."

    final = round(0.70 * skill_score + 0.20 * exp_score + 0.10 * edu_score)

    label = "🔴 Low Alignment"
    for _, (lo, hi, name) in LEVEL_WORDS.items():
        if lo <= final < hi:
            label = name

    return dict(
        role=detect_role(jd), final=final, label=label,
skill_score=round(skill_score),
        exp_score=exp_score, exp_note=exp_note, edu_score=edu_score,
edu_note=edu_note,
        required=required, preferred=preferred, matched_req=matched_req,
matched_pref=matched_pref,
        missing_req=missing_req, missing_pref=missing_pref, extra=extra,
        jd_skills=jd_skills, res_skills=res_skills,
    )


def priority_order(missing: set, required: set):
    return sorted(missing, key=lambda s: (s not in required, s))


def make_roadmap(skill: str):
    project = PROJECT_IDEAS.get(skill, f"Build a small, practical project that uses
```

```
{skill} and document it on GitHub.")
    days = [d.format(s=skill) for d in DAY_TEMPLATE]
    return days, project


def gauge(value: int):
    color = "#2ecc71" if value >= 80 else "#f1c40f" if value >= 60 else "#e67e22" if
value >= 35 else "#e74c3c"
    fig = go.Figure(go.Indicator(
        mode="gauge+number", value=value, number={"suffix": "%"},
        gauge={"axis": {"range": [0, 100]}, "bar": {"color": color},
               "steps": [{"range": [0, 35], "color": "rgba(231,76,60,0.28)"},
{"range": [35, 60], "color": "rgba(230,126,34,0.25)"},
                         {"range": [60, 80], "color": "rgba(241,196,15,0.22)"},
{"range": [80, 100], "color": "rgba(46,204,113,0.25)"}]},
        title={"text": "Resume–JD Alignment Score"}))
    fig.update_layout(height=300, margin=dict(l=20, r=20, t=60, b=10))
    return dark(fig)


def category_chart(res: dict):
    cats = {}
    for s in res["jd_skills"]:
        c = SKILLS[s][0]
        cats.setdefault(c, [0, 0])
        cats[c][1] += 1
        if s in res["res_skills"]:
            cats[c][0] += 1
    names = list(cats.keys())
    have = [cats[n][0] for n in names]
    miss = [cats[n][1] - cats[n][0] for n in names]
    fig = go.Figure()
    fig.add_bar(name="Available", x=names, y=have, marker_color="#2ecc71")
    fig.add_bar(name="Missing", x=names, y=miss, marker_color="#e74c3c")
    fig.update_layout(barmode="stack", height=320, title="Skill coverage by
category",
                      margin=dict(l=20, r=20, t=50, b=10), yaxis_title="No. of
skills")
    return dark(fig)


def build_report(res: dict) -> str:
    L = [f"CVSYNC AI · RESUME–JD ALIGNMENT REPORT", f"Role: {res['role']}",
         f"Alignment score: {res['final']}% ({res['label']})",
         "NOTE: This is resume-to-job-description alignment, NOT hiring
probability.", ""]
    L.append("PRESENT IN BOTH: " + (", ".join(sorted(res["matched_req"] |
res["matched_pref"])) or "None"))
    L.append("MISSING (required): " + (", ".join(sorted(res["missing_req"])) or
"None"))
```

```
    L.append("MISSING (preferred): " + (", ".join(sorted(res["missing_pref"])) or
"None"))
    L.append("")
    for s in priority_order(res["missing_req"] | res["missing_pref"],
res["required"]):
        days, project = make_roadmap(s)
        L.append(f"== 7-day plan: {s} ==")
        L += [f"Day {i+1}: {d}" for i, d in enumerate(days)]
        L.append(f"Mini-project: {project}")
        L.append("")
    return "\n".join(L)

# ------------------------------------------------------------
# UI
# ------------------------------------------------------------

def img_b64(name: str, mime: str) -> str:
    base = os.path.dirname(os.path.abspath(__file__))
    for p in (os.path.join(base, name), os.path.join(base, "assets", name)):
        try:
            with open(p, "rb") as f:
                return f"data:{mime};base64," + base64.b64encode(f.read()).decode()
        except Exception:
            continue
    return ""


CSS = """
<style>
#MainMenu, footer {visibility:hidden}
.stApp{background:radial-gradient(1100px 560px at 12%
-8%,rgba(37,99,235,.35),transparent),
radial-gradient(900px 500px at 100% 0%,rgba(124,58,237,.30),transparent),#070b1f}
.block-container{padding-top:2.2rem;max-width:1180px}
.nav{display:flex;justify-content:space-between;align-items:center;padding:10px 18px;
border:1px solid rgba(255,255,255,.12);border-
radius:16px;background:rgba(255,255,255,.04);margin-bottom:22px}
.brand{display:flex;align-items:center;gap:10px;font-size:1.35rem;font-
weight:700;color:#fff}
.brand img{height:38px;border-radius:8px}
.brand b,.grad{background:linear-gradient(90deg,#38bdf8,#818cf8,#c084fc);-webkit-
background-clip:text;
background-clip:text;color:transparent}
.navlinks{color:#9fb0ff;font-size:.88rem}
.pill{display:inline-block;padding:4px 14px;border:1px solid
rgba(56,189,248,.45);border-radius:20px;
color:#7dd3fc;font-size:.8rem;margin-bottom:12px}
.h1{font-size:2.7rem;font-weight:800;line-height:1.1;margin:6px 0 14px;color:#fff}
.sub{color:#b4c0ee;font-size:1.05rem;margin-bottom:18px}
.heroimg{width:100%;border-radius:20px;border:1px solid rgba(255,255,255,.18);box-
```

```
shadow:0 0 60px rgba(59,130,246,.35)}
.card{background:linear-gradient(160deg,rgba(255,255,255,.08),rgba(255,255,255,.02));
border:1px solid rgba(255,255,255,.1);border-
radius:16px;padding:18px;height:100%;min-height:150px}
.card h4{margin:6px 0;color:#fff;font-size:1.05rem}.card p{color:#aab6e6;font-
size:.88rem;margin:0}
.ico{font-size:1.7rem}
.stat{text-align:center;padding:14px;border-
radius:14px;background:rgba(255,255,255,.04);border:1px solid rgba(255,255,255,.08)}
.stat b{font-size:1.7rem;display:block;color:#7dd3fc}.stat span{color:#9fb0ff;font-
size:.82rem}
.sec{font-size:1.5rem;font-weight:700;margin:30px 0 12px;color:#fff}
.num{display:inline-block;width:30px;height:30px;line-height:30px;text-
align:center;border-radius:50%;
background:linear-gradient(90deg,#2563eb,#7c3aed);color:#fff;font-weight:700;margin-
bottom:6px}
.foot{text-align:center;color:#7f8cc4;font-size:.85rem;margin-top:34px}
div.stButton>button{border-radius:12px;border:1px solid rgba(255,255,255,.2)}
div.stButton>button[kind="primary"]{background:linear-
gradient(90deg,#2563eb,#7c3aed);border:0;color:#fff;
font-weight:600;box-shadow:0 0 24px rgba(79,70,229,.5)}
.chip{display:inline-block;padding:4px 12px;margin:3px;border-radius:16px;font-
size:.9rem;font-weight:500}
.ok{background:rgba(46,204,113,.15);color:#6ee7a1;border:1px solid
rgba(46,204,113,.5)}
.no{background:rgba(231,76,60,.15);color:#ff8a80;border:1px solid rgba(231,76,60,.5)}
.pf{background:rgba(241,196,15,.12);color:#ffd166;border:1px solid
rgba(241,196,15,.45)}
.ev{background:rgba(255,255,255,.05);border-left:4px solid #60a5fa;padding:8px
12px;margin:4px 0;
border-radius:4px;color:#dbe4ff;font-size:.9rem}
</style>
"""
st.markdown(CSS, unsafe_allow_html=True)

if "page" not in st.session_state:
    st.session_state.page = "home"


def navbar(right="Resume–JD Alignment · Skill Gap · Campus Analytics"):
    logo = img_b64("logo.png", "image/png")
    img = f"<img src='{logo}'/>" if logo else "🎯"
    st.markdown(f"<div class='nav'><div class='brand'>{img}<span>CVSync <b>AI</b>
</span></div>"
                f"<div class='navlinks'>{right}</div></div>", unsafe_allow_html=True)


if "resume_text" not in st.session_state:
    st.session_state.resume_text = ""
if "jd_text" not in st.session_state:
```

```
    st.session_state.jd_text = ""

def single_view():
    c0, c1 = st.columns([1, 4])
    with c0:
        if st.button("⚡ Load demo data"):
            st.session_state.resume_text = SAMPLE_RESUME
            st.session_state.jd_text = SAMPLE_JD
    with c1:
        st.info("Upload your resume (PDF) and paste the Job Description. Click
**Analyse**. Or press **Load demo data** to try instantly.")

    left, right = st.columns(2)
    with left:
        st.subheader("1️⃣ Student Resume")
        up = st.file_uploader("Upload resume (PDF or TXT)", type=["pdf", "txt"])
        if up is not None:
            data = up.read()
            txt = extract_pdf_text(data) if up.name.lower().endswith(".pdf") else
data.decode("utf-8", errors="ignore")
            if txt.strip():
                st.session_state.resume_text = txt
            else:
                st.warning("Could not read text from this file (maybe a scanned
image). Paste the text below instead.")
        st.session_state.resume_text = st.text_area("Resume text (editable)",
st.session_state.resume_text, height=260)

    with right:
        st.subheader("2️⃣ Job Description")
        jup = st.file_uploader("Upload JD (optional, PDF or TXT)", type=["pdf",
"txt"], key="jdup")
        if jup is not None:
            data = jup.read()
            txt = extract_pdf_text(data) if jup.name.lower().endswith(".pdf") else
data.decode("utf-8", errors="ignore")
            if txt.strip():
                st.session_state.jd_text = txt
        st.session_state.jd_text = st.text_area("Paste Job Description",
st.session_state.jd_text, height=260)

    go_btn = st.button("🔍 Analyse", type="primary", use_container_width=True)

    if go_btn:
        if not st.session_state.resume_text.strip() or not
st.session_state.jd_text.strip():
            st.error("Please provide both a resume and a job description.")
        else:
            res = analyse(st.session_state.resume_text, st.session_state.jd_text)
            if not res["jd_skills"]:
```

```
                st.warning("No known skills detected in the JD. Try a more detailed
JD.")
            st.divider()
            st.header(f"Result for: {res['role']}")
            a, b = st.columns([1, 1])
            with a:
                st.plotly_chart(gauge(res["final"]), use_container_width=True)
                st.subheader(res["label"])
            with b:
                st.plotly_chart(category_chart(res), use_container_width=True)

            m1, m2, m3 = st.columns(3)
            m1.metric("Skills alignment", f"{res['skill_score']}%")
            m2.metric("Experience alignment", f"{res['exp_score']}%")
            m3.metric("Education alignment", f"{res['edu_score']}%")
            st.caption(f"📌 {res['exp_note']}  \n🎓 {res['edu_note']}")

            st.warning("⚠️ This score shows **resume-to-job-description alignment
only**. "
                       "It is **not** a prediction of hiring probability or a
guarantee of selection.")

            tab1, tab2, tab3, tab4 = st.tabs(["✅ Skill Match", "🔎 Resume Evidence",
"🗺️ Gap-to-Roadmap", "💡 Extras & Tips"])

            with tab1:
                st.markdown("**Present in both JD and resume**")
                have = sorted(res["matched_req"] | res["matched_pref"])
                st.markdown("".join(f"<span class='chip ok'>✔ {s}</span>" for s in
have) or "_None_", unsafe_allow_html=True)
                st.markdown("**Required by company, missing from resume**")
                st.markdown("".join(f"<span class='chip no'>✘ {s}</span>" for s in
sorted(res["missing_req"])) or "_None 🎉_",
                            unsafe_allow_html=True)
                if res["missing_pref"]:
                    st.markdown("**'Nice-to-have' skills missing**")
                    st.markdown("".join(f"<span class='chip pf'>○ {s}</span>" for s
in sorted(res["missing_pref"])),
                                unsafe_allow_html=True)
                st.markdown("**Technical skill-gap table**")
                rows = []
                for s in sorted(res["jd_skills"]):
                    status = "Available" if s in res["res_skills"] else "Missing"
                    kind = "Required" if s in res["required"] else "Preferred"
                    rows.append({"Skill": s, "Category": SKILLS[s][0], "Type": kind,
"Status": status})
                st.dataframe(rows, use_container_width=True, hide_index=True)

            with tab2:
                st.caption("Proof lines from the resume for every matched skill.")
```

```
                matched_all = sorted(res["matched_req"] | res["matched_pref"])
                if not matched_all:
                    st.info("No matching skills found, so no evidence to show.")
                for s in matched_all:
                    st.markdown(f"**{s}**")
                    for line in evidence_for(s, st.session_state.resume_text):
                        st.markdown(f"<div class='ev'>“{line}”</div>",
unsafe_allow_html=True)
                weak = [s for s in matched_all if len(evidence_for(s,
st.session_state.resume_text, 5)) == 1
                        and "skills" in evidence_for(s, st.session_state.resume_text,
1)[0].lower()]
                if weak:
                    st.warning("These skills are only listed in a skills list with no
project or experience proof: "
                               + ", ".join(weak) + ". Add a project line showing how
you used them.")

            with tab3:
                gaps = priority_order(res["missing_req"] | res["missing_pref"],
res["required"])
                if not gaps:
                    st.success("No gaps found. Your resume covers all detected JD
skills!")
                for s in gaps:
                    days, project = make_roadmap(s)
                    tag = "Required" if s in res["required"] else "Nice-to-have"
                    with st.expander(f"{s}  ·  {tag}", expanded=(s == gaps[0])):
                        for i, d in enumerate(days, 1):
                            st.markdown(f"**Day {i}:** {d}")
                        st.success(f"🛠️ Mini-project to add to resume: {project}")

            with tab4:
                st.markdown("**Extra skills you have that the JD did not ask for**
(still good to keep)")
                st.markdown("".join(f"<span class='chip pf'>{s}</span>" for s in
sorted(res["extra"])) or "_None_",
                            unsafe_allow_html=True)
                st.markdown("**Suggested improvements**")
                tips = []
                if res["missing_req"]:
                    tips.append(f"Focus first on: {',
'.join(priority_order(res['missing_req'], res['required'])[:3])}.")
                tips.append("Add numbers to project bullets (accuracy, users, time
saved).")
                tips.append("Use the same keywords as the JD where they are
truthfully applicable.")
                tips.append("Move skills with proof (projects) above skills with
none.")
                for t in tips:
```

```
                    st.markdown(f"- {t}")

            st.download_button("⬇️ Download report (TXT)", build_report(res),
file_name="cv_sync_report.txt")



def batch_view():
    import csv
    st.subheader("🏫 Campus Batch Analysis")
    st.caption("Upload many resumes against one JD. Students are anonymised (Student
1, 2, 3...) so the dashboard shows only batch-level skill gaps.")
    st.session_state.jd_text = st.text_area("Job Description for this batch",
st.session_state.jd_text, height=180, key="batch_jd")
    files = st.file_uploader("Upload multiple resumes (PDF/TXT)", type=["pdf",
"txt"], accept_multiple_files=True, key="batch_files")
    use_sample = st.button("⚡ Use 6 sample students + sample JD")
    if use_sample:
        st.session_state.batch_sample = True
        if not st.session_state.jd_text.strip():
            st.session_state.jd_text = SAMPLE_JD
            st.rerun()
    if files:
        st.session_state.batch_sample = False

    resumes = []
    if files:
        for f in files:
            data = f.read()
            t = extract_pdf_text(data) if f.name.lower().endswith(".pdf") else
data.decode("utf-8", errors="ignore")
            resumes.append(t)
    elif st.session_state.get("batch_sample"):
        resumes = SAMPLE_BATCH

    if not resumes:
        st.info("Upload resumes or press the sample button.")
        return
    if not st.session_state.jd_text.strip():
        st.error("Please paste a Job Description.")
        return

    results = [analyse(t, st.session_state.jd_text) for t in resumes]
    n = len(results)
    avg = round(sum(r["final"] for r in results) / n)
    ready = sum(1 for r in results if r["final"] >= 60)

    k1, k2, k3 = st.columns(3)
    k1.metric("Students analysed", n)
    k2.metric("Average alignment", f"{avg}%")
```

```
    k3.metric("Strong / Good alignment", f"{ready} of {n}")

    jd_all = sorted(results[0]["jd_skills"])
    gap_rows = []
    for s in jd_all:
        have = sum(1 for r in results if s in r["res_skills"])
        gap_rows.append((s, round((n - have) / n * 100), results[0]
["required"].__contains__(s)))
    gap_rows.sort(key=lambda x: -x[1])

    fig = go.Figure(go.Bar(x=[g[1] for g in gap_rows], y=[g[0] for g in gap_rows],
orientation="h",
                           marker_color=["#e74c3c" if g[1] >= 60 else "#e67e22" if
g[1] >= 30 else "#2ecc71" for g in gap_rows]))
    fig.update_layout(title="% of students MISSING each skill", xaxis_title="% of
batch missing",
                      height=max(320, 28 * len(gap_rows) + 100),
yaxis=dict(autorange="reversed"),
                      margin=dict(l=20, r=20, t=50, b=10))
    st.plotly_chart(dark(fig), use_container_width=True)

    top = [g for g in gap_rows if g[1] >= 50]
    if top:
        st.warning("🎯 **Training recommendation:** more than half the batch lacks "
+ ", ".join(g[0] for g in top[:5]) +
                   ". A workshop on these would benefit the most students.")

    hist = go.Figure(go.Histogram(x=[r["final"] for r in results], nbinsx=10,
marker_color="#4a6cf7"))
    hist.update_layout(title="Distribution of alignment scores", xaxis_title="Score
(%)", yaxis_title="Students",
                       height=280, margin=dict(l=20, r=20, t=50, b=10))
    st.plotly_chart(dark(hist), use_container_width=True)

    order = sorted(range(n), key=lambda i: -results[i]["final"])
    rows = [{"Student": f"Student {i+1}", "Alignment %": results[i]["final"],
"Level": results[i]["label"],
             "Required skills missing": len(results[i]["missing_req"]),
             "Top gaps": ", ".join(sorted(results[i]["missing_req"])[:4])} for i in
order]
    st.markdown("**Ranked list (anonymised)**")
    st.dataframe(rows, use_container_width=True, hide_index=True)

    buf = io.StringIO()
    w = csv.DictWriter(buf, fieldnames=list(rows[0].keys()))
    w.writeheader()
    w.writerows(rows)
    st.download_button("⬇️ Download batch report (CSV)", buf.getvalue(),
file_name="cv_sync_batch_report.csv")
    st.caption("Scores show resume-to-JD alignment only, not hiring probability.
```

```
Sample data is used only when the sample button is pressed.")



HOME_CSS = """
<style>
.stApp{background:
radial-gradient(circle at 50% 30%,rgba(255,235,59,.55),transparent 55%),
repeating-conic-gradient(from 0deg at 50% 30%,#f97316 0deg 9deg,#facc15 9deg
18deg,#16a34a 18deg 27deg,#ef4444 27deg 36deg,#fb923c 36deg 45deg)}
.block-container{padding-top:1.5rem}
.heroimg{border:4px solid #111;border-radius:18px;box-shadow:10px 10px 0
#111;width:100%}
.tag{display:inline-block;background:#111;color:#fff;font-weight:700;padding:8px
18px;border-radius:12px;
box-shadow:4px 4px 0 #0008;margin:6px 0}
.sec{display:inline-block;background:#111;color:#facc15;padding:6px 16px;border-
radius:10px;
box-shadow:4px 4px 0 #0008;margin:30px 0 14px}
.card{background:#111827;border:3px solid #111;box-shadow:6px 6px 0 #111;color:#fff}
.card h4{color:#facc15}.card p{color:#e5e7eb}
.stat{background:#111827;border:3px solid #111;box-shadow:5px 5px 0 #111}
.stat b{color:#facc15}.stat span{color:#e5e7eb}
.num{background:#facc15;color:#111}
.foot{background:#111;color:#fff;padding:10px 16px;border-radius:12px;display:inline-
block}
div.stButton>button{border:3px solid #111;box-shadow:4px 4px 0 #111;font-
weight:800;border-radius:12px;color:#111;background:#fff}
div.stButton>button[kind="primary"]{background:#facc15;color:#111;border:3px solid
#111;box-shadow:4px 4px 0 #111}
div.stButton>button:hover{transform:translate(-1px,-1px)}
</style>
"""


def home_view():
    st.markdown(HOME_CSS, unsafe_allow_html=True)
    hero = img_b64("hero.jpg", "image/jpeg")
    if hero:
        st.markdown(f"<img class='heroimg' src='{hero}'/>", unsafe_allow_html=True)
    else:
        st.markdown("<div class='h1'>CVSync <span class='grad'>AI</span></div>",
unsafe_allow_html=True)

    st.markdown("<div style='text-align:center'><span class='tag'>AI Resume–Job
Matching &amp;amp; Skill Gap Analyzer "
                "· INTELLIQ SOLVEX · Dept. of AI&amp;amp;DS</span></div>",
unsafe_allow_html=True)
    _, mid, _ = st.columns([1, 2, 1])
    with mid:
```

```
        if st.button("🚀 Get Started", type="primary", use_container_width=True):
            st.session_state.page = "app"
            st.rerun()
        if st.button("⚡ Try with demo data", use_container_width=True):
            st.session_state.resume_text = SAMPLE_RESUME
            st.session_state.jd_text = SAMPLE_JD
            st.session_state.page = "app"
            st.rerun()

    st.markdown("<div style='height:14px'></div>", unsafe_allow_html=True)
    for col, (v, t) in zip(st.columns(4), [(f"{len(SKILLS)}+", "skills tracked"),
("7-day", "learning roadmaps"),
                                           ("Batch", "anonymised analytics"), ("0",
"setup, opens in browser")]):
        col.markdown(f"<div class='stat'><b>{v}</b><span>{t}</span></div>",
unsafe_allow_html=True)

    st.markdown("<div class='sec'>What CVSync AI does</div>", unsafe_allow_html=True)
    cards = [("🎯", "Alignment Score", "Resume-to-JD alignment with required and
nice-to-have skills weighed separately."),
             ("🔎", "Resume Evidence", "Shows the exact resume line that proves each
skill, and flags skills with no proof."),
             ("🗺️", "Gap-to-Roadmap", "A 7-day plan and a mini-project idea for every
missing skill."),
             ("🏫", "Campus Batch View", "Upload many resumes and see which skills
the whole batch lacks, without student names.")]
    for col, (i, h, p) in zip(st.columns(4), cards):
        col.markdown(f"<div class='card'><div class='ico'>{i}</div><h4>{h}</h4><p>{p}
</p></div>", unsafe_allow_html=True)

    st.markdown("<div class='sec'>How it works</div>", unsafe_allow_html=True)
    steps = [("1", "Upload", "Add the student resume (PDF) and paste the job
description."),
             ("2", "Analyse", "Skills, experience and education are matched, with
synonyms like ML = Machine Learning."),
             ("3", "Improve", "Follow the roadmap, or use batch mode to plan campus
training.")]
    for col, (n, h, p) in zip(st.columns(3), steps):
        col.markdown(f"<div class='card'><span class='num'>{n}</span><h4>{h}</h4><p>
{p}</p></div>", unsafe_allow_html=True)

    st.markdown("<div style='text-align:center;margin-top:30px'><span class='foot'>⚠️
The score shows resume-to-job-description "
                "alignment only, not hiring probability.</span></div>",
unsafe_allow_html=True)


def app_view():
    navbar("Resume–JD Alignment · Skill Gap · Campus Analytics")
    if st.button("← Home"):
```

```
        st.session_state.page = "home"
        st.rerun()
    mode = st.radio("Mode", ["👤 Single Resume", "🏫 Campus Batch"], horizontal=True)
    if mode.startswith("👤"):
        single_view()
    else:
        batch_view()
    st.markdown("<div class='foot'>CVSync AI · INTELLIQ SOLVEX · Real Campus
Problems. Intelligent Solutions.</div>",
                unsafe_allow_html=True)


if st.session_state.page == "home":
    home_view()
else:
    app_view()
```